Jugador Inteligente para el Juego HEX

Búsqueda Adversarial y Monte Carlo Tree Search

Autora: Olivia Ibañez Mustelier
Fecha: Abril 2025
Repositorio: `Proyecto_IA_HEX-main`

Resumen

Este trabajo presenta la implementación y evaluación comparativa de cuatro estrategias de inteligencia artificial para el juego de tablero HEX: un jugador aleatorio, un jugador greedy, un jugador basado en Minimax con poda alfa-beta y un jugador basado en Monte Carlo Tree Search (MCTS). Se diseñaron heurísticas específicas del dominio basadas en la teoría de puentes y caminos conectivos. Los experimentos, realizados en tableros 7x7 con 20 partidas por enfrentamiento (120 partidas totales), muestran que MCTS supera estadísticamente a Minimax (16-4, p=0.012), y que ambos dominan a los baselines aleatorio y greedy. Sorprendentemente, no se observó ventaja significativa del primer jugador cuando ambos oponentes emplean algoritmos de búsqueda adversarial (49.2% vs 50.8%, p=0.927). El trabajo demuestra que MCTS, con un presupuesto de tiempo modesto (0.5s por jugada), es capaz de superar a Minimax con profundidad 2, validando su eficacia en dominios con horizonte de planificación largo.

Palabras clave: HEX, Minimax, poda alfa-beta, Monte Carlo Tree Search, heurísticas, puentes, búsqueda adversarial.

---

1. Introducción

1.1 Motivación

El juego HEX, inventado independientemente por Piet Hein (1942) y John Nash (1948), es un juego de tablero para dos jugadores con reglas extremadamente simples pero con una profundidad estratégica notable. Se juega en un tablero romboidal de celdas hexagonales, donde cada jugador intenta conectar sus dos lados opuestos mediante una cadena de fichas contiguas.

A pesar de su simplicidad, HEX es **PSPACE-completo** (Reisch, 1981), lo que lo convierte en un excelente banco de pruebas para algoritmos de búsqueda adversarial. Además, HEX tiene propiedades matemáticas elegantes: no puede haber empates y el primer jugador siempre tiene una estrategia ganadora (teorema de estrategia de Nash), aunque no se conoce explícitamente para tableros grandes.

1.2 Objetivos

1. Implementar cuatro estrategias de IA para HEX.
2. Diseñar heurísticas específicas del dominio.
3. Comparar experimentalmente su rendimiento con rigor estadístico.
4. Analizar la influencia del tamaño del tablero y la ventaja del primer jugador.

1.3 Contribuciones

- Una implementación modular y eficiente del juego con NumPy.
- Cuatro jugadores con distintos niveles de sofisticación.
- Heurísticas basadas en puentes y caminos conectivos.
- Un protocolo experimental con tests binomiales para validar resultados.
- Un análisis crítico de la ventaja del primer jugador.

---

2. Estado del arte

2.1 Minimax con poda alfa-beta

El algoritmo Minimax (von Neumann, 1928) es la base de la búsqueda adversarial en juegos de suma cero. Explora el árbol de juego asumiendo que ambos jugadores juegan óptimamente. La poda alfa-beta (Knuth & Moore, 1975) reduce el número de nodos explorados sin alterar el resultado.

Complejidad: O(b^d) sin poda, O(b^(d/2)) con poda en el mejor caso, donde b es el factor de ramificación y d la profundidad.

Limitaciones: Requiere una función de evaluación heurística y sufre de "miopía" si la profundidad es baja.

2.2 Monte Carlo Tree Search (MCTS)

MCTS (Coulom, 2006; Browne et al., 2012) es un método de búsqueda que combina simulación aleatoria con selección basada en UCB (Upper Confidence Bound). Sus cuatro fases son: selección, expansión, simulación y retropropagación.

Ventajas:

- No requiere función de evaluación explícita.
- Asimétrico: explora más donde promete.
- Converge a la jugada óptima con suficiente tiempo.

Aplicaciones: AlphaGo (Silver et al., 2016), AlphaZero (Silver et al., 2018).

2.3 Heurísticas específicas de HEX

En HEX, las nociones de **puente** y **conexión virtual** son fundamentales. Un puente es una conexión indirecta entre dos fichas del mismo jugador, con dos celdas intermedias. Si el oponente ocupa una, el jugador puede ocupar la otra para mantener la conexión.

Referencias: Hayward et al. (2007), Anshelevich (2002), Yang et al. (2016).

---

3. Metodología

3.1 Representación del tablero

El tablero se representa como una matriz `numpy.ndarray` de tamaño N×N con valores:

- `0`: celda vacía
- `1`: ficha del jugador 1 (conecta filas 0 y N-1)
- `2`: ficha del jugador 2 (conecta columnas 0 y N-1)

Los vecinos se definen mediante desplazamiento *odd-r* (filas impares desplazadas a la derecha):

DIRECCIONES = [(-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0)]

3.2 Jugadores implementados

Jugador	Descripción	Estrategia
`RandomPlayer`	Baseline aleatorio	Elige celda vacía al azar
`GreedyPlayer`	Baseline voraz	Maximiza heurística local
`MinimaxPlayer`	Búsqueda adversarial	Minimax + alfa-beta + heurísticas
`MCTSPlayer`	Búsqueda estocástica	MCTS con rollout heurístico (80% greedy, 20% random)

3.3 Heurísticas

1. **Puentes pendientes:** celdas vacías que completarían un puente entre dos fichas propias.
2. **Puentes en riesgo:** celdas donde un puente propio tiene una salida ocupada por el oponente.
3. **Conectividad a bordes:** evaluación de caminos que conectan los bordes objetivo.
4. **Proximidad:** puntuación basada en vecinos propios y del oponente.

3.4 Función de evaluación

La heurística global suma las puntuaciones de todas las celdas vacías, con bonus por completar puentes (+20) y por bloquear puentes en riesgo (+40). La victoria se valora en ±100,000.

3.5 Protocolo experimental

- **Tamaño del tablero:** 7×7 (principal) y 11×11 (exploratorio).
- **Partidas por enfrentamiento:** 20 (7×7), 4 (11×11).
- **Alternancia:** el orden se alterna para eliminar sesgos.
- **Test estadístico:** binomial bilateral con H_0 = 50/50.
- **Significancia:** $\alpha = 0.05$.

---

4. Experimentos y resultados

4.1 Torneo 7×7 (20 partidas por enfrentamiento)

Tabla 1. Resultados del torneo.

Enfrentamiento	Victorias (J1-J2)	p-value	Significativo
Random vs Greedy	9-11	0.824	No
Random vs Minimax_d2	2-18	0.0004	Sí
Random vs MCTS_05s	0-20	<0.0001	Sí
Greedy vs Minimax_d2	0-20	<0.0001	Sí
Greedy vs MCTS_05s	0-20	<0.0001	Sí

Enfrentamiento	Victorias (J1-J2)	p-value	Significativo
Minimax_d2 vs MCTS_05s	**4-16**	**0.012**	**Sí**

Tabla 2. Ranking final.

Jugador	Victorias	Win rate
MCTS_05s	56	62.2%
Minimax_d2	42	46.7%
Random	11	12.2%
Greedy	11	12.2%

Tabla 3. Ventaja del primer jugador.

Métrica	Valor
Primer jugador	59 (49.2%)
Segundo jugador	61 (50.8%)
p-value	0.927
Significativo	No

Figuras:

- `figs/wins.png`: victorias totales por jugador.
- `figs/times.png`: tiempo medio por partida.
- `figs/first_player_advantage.png`: ventaja estructural.

4.2 Experimento exploratorio 11×11

Tabla 4. Minimax_d2 vs MCTS_05s en tablero 11×11 (4 partidas).

Enfrentamiento	Victorias (J1-J2)	Primer jugador	Segundo jugador
Minimax_d2 vs MCTS_05s	1-3	1	3

Tiempo medio por partida: 26.7 segundos.

El experimento confirma la tendencia observada en 7×7: **MCTS mantiene su superioridad sobre Minimax al escalar el tamaño del tablero**, aunque el número reducido de partidas (limitado por el coste computacional) impide alcanzar significancia estadística.

Nota metodológica: El tiempo medio por partida en 11×11 es ~4× mayor que en 7×7, lo que motiva la reducción del número de partidas en este experimento exploratorio.

---

5. Discusión

5.1 MCTS supera a Minimax

El hallazgo principal es que **MCTS con 0.5s por jugada supera estadísticamente a Minimax con profundidad 2 y 1.5s por jugada** (16-4, p=0.012). Este resultado es notable por varias razones:

1. **Horizonte de planificación:** MCTS explora cientos de partidas completas por jugada, mientras que Minimax_d2 solo mira 2 niveles. En HEX, donde una jugada puede tener consecuencias a 10+ turnos, este horizonte largo es crítico.
2. **Rollout heurístico:** El uso de una política greedy con 80% de probabilidad durante la simulación permite a MCTS aprovechar el conocimiento del dominio sin depender exclusivamente de él.
3. **Presupuesto de tiempo:** MCTS adapta su esfuerzo al tiempo disponible. Con 0.5s, realiza ~500 iteraciones por jugada, suficientes para estimar el valor de cada acción.
4. **Asimetría:** MCTS dedica más esfuerzo a las ramas prometedoras (gracias a UCB), mientras que Minimax explora todas las ramas uniformemente hasta la profundidad fijada.

5.2 Greedy no mejora sobre Random

Sorprendentemente, **Greedy no supera a Random** (9-11, p=0.824). Esto sugiere que en tableros pequeños, una heurística puramente local (basada en proximidad de vecinos) es insuficiente. La evaluación global (puentes, caminos) que usan Minimax y MCTS es indispensable.

5.3 Ausencia de ventaja del primer jugador

A pesar de la creencia clásica de que el primer jugador tiene ventaja en HEX (garantizada por el teorema de estrategia), **no se observó ventaja significativa** cuando ambos jugadores usan algoritmos de búsqueda adversarial. Esto se explica porque:

- Los algoritmos inteligentes compensan la desventaja estructural explotando errores del oponente.
- En tableros pequeños, la ventaja del primero es menor.
- El número de partidas (120) es suficiente para detectar ventajas moderadas (>55%).

Este hallazgo contrasta con experimentos previos Random vs Random donde la ventaja era de ~60%, sugiriendo que **la ventaja del primero depende fuertemente del nivel de los jugadores**.

5.4 Limitaciones

1. **Tablero 7×7:** Aunque válido para comparación, no es el tamaño estándar de HEX (11×11).
2. **MCTS secuencial:** Sin paralelización, el tiempo por jugada limita las iteraciones.
3. **Sin swap:** No se implementó la regla del "pie" que elimina la ventaja del primero.
4. **Heurísticas fijas:** Los pesos (+20, +40, etc.) no fueron optimizados.
5. **Experimento 11×11 con pocas partidas:** Limitado por coste computacional.

---

6. Conclusiones y trabajo futuro

6.1 Conclusiones

1. **MCTS es el mejor jugador** en el experimento 7×7 (62.2% win rate).

2. **Minimax con profundidad 2 y heurísticas es competitivo** pero inferior a MCTS.
3. **Las heurísticas de puentes son críticas:** sin ellas, ni siquiera Greedy supera a Random.
4. **La ventaja del primer jugador desaparece** con jugadores inteligentes.
5. **La tendencia se mantiene en 11×11** (MCTS 3-1 Minimax), aunque con menos datos.

6.2 Trabajo futuro

1. **Escalar a 11×11** con MCTS paralelizado.
2. **Implementar AlphaZero:** MCTS + red neuronal para sustituir el rollout heurístico.
3. **Optimizar heurísticas** mediante aprendizaje por refuerzo o algoritmos genéticos.
4. **Implementar la regla del swap** para equilibrar el juego.
5. **Interfaz web avanzada** con visualización hexagonal real y análisis en vivo.

---

7. Referencias

1. Anshelevich, V. (2002). *The Game of Hex: The Hierarchical Approach*. In More Games of No Chance.
2. Browne, C., et al. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games.
3. Coulom, R. (2006). *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. Computers and Games.
4. Hayward, R., et al. (2007). *The Game of Hex: A Survey*. Technical Report.
5. Knuth, D., & Moore, R. (1975). *An Analysis of Alpha-Beta Pruning*. Artificial Intelligence.
6. Reisch, S. (1981). *Hex is PSPACE-complete*. Acta Informatica.
7. Silver, D., et al. (2016). *Mastering the Game of Go with Deep Neural Networks and Tree Search*. Nature.
8. Silver, D., et al. (2018). *A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play*. Science.
9. von Neumann, J. (1928). *Zur Theorie der Gesellschaftsspiele*. Mathematische Annalen.
10. Yang, J., et al. (2016). *Deep Learning for Hex*. arXiv preprint.

Apéndice A: Estructura del código

src/hex_ai/ ├── board.py # Representación del tablero ├── game.py # Lógica del juego ├── heuristics.py # Heurísticas de puentes y caminos ├── evaluation.py # Funciones de enfrentamiento ├── utils.py # Utilidades (puentes, vecinos) └── players/ ├── base.py # Clase abstracta ├── random_player.py ├── greedy_player.py ├── minimax_player.py └── mcts_player.py

Apéndice B: Reproducibilidad

```
# Instalar dependencias
pip install -r requirements.txt

# Ejecutar torneo
python -m experiments.run_tournament
python -m experiments.analyze_results
```

```
# Experimentos exploratorios
jupyter notebook notebooks/exploratory.ipynb
```